

OptSem-C: Benchmarking Projection Precision in Public Query-Optimizer Contracts

Haoyi Zhang

Xi'an Jiaotong-Liverpool University
Suzhou, China
hyeliozhang@gmail.com

Xunzhu Tang

University of Luxembourg
Luxembourg, Luxembourg
xunzhu.tang@uni.lu

Huaijin Ran

Nanyang Technological University
Singapore, Singapore
huaijin003@e.ntu.edu.sg

Tegawendé F. Bissyandé

University of Luxembourg
Luxembourg, Luxembourg
tegawende.bissyande@uni.lu

ABSTRACT

Optimizer comparisons in database papers often start with public labels such as pushdown, join, or cost-based optimization. Those labels are readable, but they are not equality predicates for query-processing behavior. A BigQuery stage label, a Trino connector-pushdown claim, a local rewrite, and a runtime filter can all be summarized as “pushdown” even though they support different comparison conclusions. OptSem-C studies this data-management denominator before latency, cardinality, or plan-quality measurements are read. It encodes each source-supported optimizer claim as a guarded public contract over action, query guard, optimizer layer, execution placement, decision time, observability, and evidence support, then asks which public distinctions a comparison vocabulary preserves.

The audit is fixed before witness counting: schema, sources, projections, motif crosswalk, and repair-field universe are all replay inputs. On seven analytical SQL engines, 287 source-linked rules, 4,216 executable probes, and 26 observed comparison surfaces, keyword, operator-only, and yes/no vocabularies induce 254, 238, and 6 contract-collision witnesses. These witnesses occupy four observed engine pairs and 486 distinct probes; they are denominator errors over public contracts, not private optimizer bug claims or population estimates for future engines. The benchmark also covers 90 published optimizer-motif requirements through 71 distinct SQL representatives. Placement repairs the keyword collisions, optimizer layer repairs the operator-only collisions, and layer plus placement repairs all 498 headline witnesses. The result is a replayable public-contract audit: comparison papers can check which optimizer distinction they collapsed, which retained fields repair it, and where the finite evidence stops.

PVLDB Reference Format:

Haoyi Zhang, Huaijin Ran, Xunzhu Tang, and Tegawendé F. Bissyandé.
OptSem-C: Benchmarking Projection Precision in Public Query-Optimizer Contracts. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://doi.org/10.5281/zenodo.21211651>. Archive SHA-256: 1e03ab573a75882b111e25db50500ce2c6be88f7313b90efd19b066523c8275a.

1 INTRODUCTION

SQL deliberately hides physical execution choices. That abstraction is right for applications, but it becomes dangerous when the hidden choice is the object being compared in a database study. A benchmark author may ask whether two engines support pushdown, a system integrator may compare the evidence available in plan output, and a database researcher may group systems by join or adaptivity labels. The same SQL query can then expose different public behavior: a local predicate rewrite, connector-side execution, a service-stage graph, a runtime filter, or only a logical explanation. Before any latency or plan-quality number is interpreted, the comparison needs to know whether those public query-processing behaviors are being treated as the same object.

The problem is easiest to see in a concrete audited case. For probe P0009, a keyword view equates four source records under a pushdown/explain label: BigQuery federated filter delegation, BigQuery query-stage evidence, Trino connector pushdown, and Trino runtime dynamic filtering. The public contracts do not say the same thing: they distinguish connector-source delegation, service-stage evidence, aggregate/projection pushdown, and runtime filtering. A paper that keeps only the keywords has not found an optimizer bug; it has lost the denominator needed to decide whether two public claims were comparable in the first place.

This is not a straw-man vocabulary. We audit 26 public comparison surfaces drawn from engine documentation, plan interfaces, and optimizer-control descriptions. All expose keyword-style or operator-family labels, twelve expose yes/no-style controls, and none exposes the full reference payload used by OptSem-C. The work therefore asks a narrow systems question: when public optimizer behavior is summarized at the vocabulary level exposed by comparison surfaces, which source-supported distinctions disappear, and which retained fields make the comparison auditable again?

Database research already has precise abstractions for the objects it chooses to study. Relational semantics starts from Codd’s model [20]; Selinger-style optimizers separate access-path search

from costing [60]; Volcano exposes rule-driven and parallel query processing [30]; the Volcano optimizer-generator account makes extensible search explicit [32]; and Cascades turns that line into a task-driven framework [31]. Chaudhuri surveys optimizer architecture [14]; Ioannidis surveys cost and cardinality assumptions [38]; and Steinbrunn et al. study join-order search heuristics [63]. Public optimizer behavior is usually compared with much coarser vocabularies. A cell labeled “pushdown” may mean predicate movement inside the engine, partition pruning, aggregation delegation, join delegation, limit delegation, or runtime filtering. A cell labeled “join” may refer to a logical operator, an estimated physical operator, a distributed exchange, or a runtime profile entry. A claim that two engines are “cost based” may hide whether statistics are local, connector-provided, unavailable, or revised during execution. These shortcuts are not harmless summarization: they can create projection-induced contract collisions (a coarse vocabulary declares equality among public contracts whose reference signatures differ).

OptSem-C treats this precision problem as a query-processing systems problem, not as prose cleanup. A *public optimizer contract* is the optimizer behavior an engine exposes through plan interfaces, tuning surfaces, and system descriptions. A source record can support a contract that is not visible in a particular textual plan, and a plan tag can expose behavior without proving the private cost model. The object is therefore not a hidden implementation trace and not a latency measurement. OptSem-C represents each public statement as a guarded contract rule over query features. The rule records the action, the guard under which it applies, where the action is placed, which optimizer layer owns it, when the decision is made, how the behavior is observable, and whether the public contract requires, permits, forbids, or does not specify the action.

Research questions. We ask four finite, auditable questions. RQ1 asks whether public optimizer claims can be encoded as source-linked contracts and executable probes without becoming a latency benchmark. RQ2 asks how often comparison vocabularies that appear in public feature claims collapse distinct source-supported contracts. RQ3 asks which semantic fields eliminate those collisions, and which robustness checks delimit the repair claim. RQ4 asks whether the projection-audit inner loop and SQL-denominator checks are cheap enough to rerun when the public source denominator changes, rather than trusted as a one-off manual inspection.

The central result is a projection-kernel account of these contract collisions. Comparing optimizer contracts through a keyword or operator vocabulary applies a predeclared projection to reference signatures: the full public evidence payload inside the declared schema, not an oracle over private optimizer state. If reference signatures disagree but fall into the same projected class, the projection has declared an equivalence that source-supported public signatures do not support. Contract collisions are therefore denominator errors for the comparison itself; performance or cost measurements should be interpreted only after the compared behavior has been named.

OptSem-C turns that denominator question into a finite comparison relation with executable counter-witnesses, exact repair certificates, and negative controls. The contract schema, source manifest, projection portfolio, motif crosswalk, and repair-field universe are frozen before witness counting; execution checks validate probes but never admit rules or choose repairs. Layer and

placement are therefore not learned explanations for the failures: they are schema dimensions inherited from optimizer architecture and federation literature, then tested in the same predeclared field universe as every other retained field. Keyword projection declares 2,284 engine-probe equivalences: 2,030 have matching reference signatures and 254 are separated by reference signatures. Operator-only projection declares 2,268 equivalences, including 238 collisions; the yes/no projection contributes six sparse source-supported records. Across the three vocabularies, the total is 498 projection-witness records over four observed engine pairs and 486 distinct probes, where a record is keyed by projection, engine pair, and probe. Restoring placement repairs keyword collisions; restoring optimizer layer repairs operator-only collisions; retaining layer and placement is one interpretable two-field repair basis for all headline records within the admitted public-contract corpus.

The robustness checks are designed to block three tempting overclaims. First, the probe set is a rule-aware coverage benchmark, not an independent population sample, so we do not report statistical generalization beyond the frozen denominator. Second, source-removal is an information-deletion stress rather than a new sample: keyword and operator-only remain nonzero after every single-source removal, and the audit separately records retained original witnesses and newly induced witnesses. The sparse yes/no projection is reported as source-sensitive and stays outside the broad robustness claim. Third, repair fields are not learned from failed engine-pair transfer: the reported layer-and-placement basis resolves feature-family and engine-family stress splits, while point-learned engine-pair repairs fail on held-out pairs and remain a boundary. A field-only stress test over all 256 retained-field vocabularies leaves only 44 unsafe regions, all covered by concrete counter-witnesses; after excluding action-variant identifiers, no singleton semantic field is safe.

We also introduce OptSemBench-C, a covering benchmark for optimizer-contract behavior. Its denominator is not raw query count. It is coverage of optimizer-relevant feature interactions: source boundary, connector capability, join shape, predicate class, aggregation, statistics need, reuse, distribution, adaptivity, and control surface. The generated probes are executable SQL representatives. The probe space is then cross-checked against DPccp-style join enumeration [51], JOB optimizer evidence [45], later JOB diagnostics [46], exact-cardinality testing [15], progressive optimization [49], mid-query reoptimization [41], adaptive query processing [26], and eddies [7]. We use this audit as a published-motif vocabulary stress test: 90 optimizer-motif requirements are covered by 71 distinct generated SQL representatives, so the check is concrete without pretending that generated representatives are copied JOB, TPC-H, or TPC-DS workloads.

Contributions. C1 defines a public optimizer contract as a query-processing object separate from SQL result equivalence and runtime performance; Section 3 gives the guarded rule model and the state-join semantics. C2 turns comparison validity into a finite projection audit: reference signatures define the public evidence relation, projected signatures expose counter-witnesses, and repair certificates identify fields that make a vocabulary safe. C3 contributes OptSemBench-C as a denominator design for optimizer-contract comparisons across seven analytical SQL engines; Section 4 reports source-linked admission, executable probes, external-motif

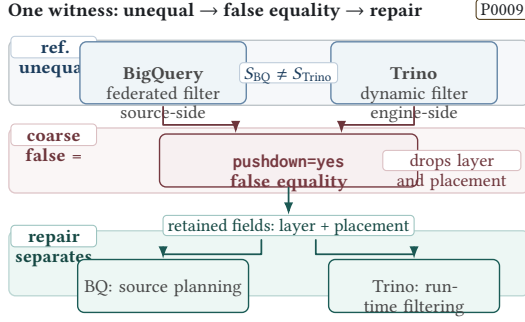


Figure 1: Projection loss as one finite contract witness from the 498 headline witness records. BigQuery federated filtering and Trino dynamic filtering differ under the reference signature, collapse under a coarse pushdown=yes denominator, and separate after retaining layer and placement.

coverage, and zero-failure admission checks. C4 reports the projection kernels, anti-overfitting checks, resource profile, and repair certificates: 498 headline projection-witness records, layer and placement as an interpretable repair basis for those records, and cloud replay evidence for the comparison inner loop.

Each central claim is tied to a reproducible record rather than to prose alone. The model section defines the state table and projection obligations; the corpus section exposes source identifiers, line spans, and digest prefixes; the evaluation tables connect SQL execution, motif coverage, collision counts, and repair certificates to regenerated outputs. The case studies then show how those aggregate certificates change comparison judgments.

2 PRECISION LOSS IN OPTIMIZER COMPARISONS

Figure 1 gives the smallest example. A feature matrix maps several public optimizer contracts into the same label. After the mapping, behaviors with different placement, action kind, and evidence layer become indistinguishable, so a study that treats the label as equality can compare non-equivalent public optimizer behavior. The issue is prior to workload choice: JOB or TPC-H can standardize queries, but they do not say whether two engines expose the same public pushdown, adaptation, or plan-evidence contract.

Feature-name collision. A cell such as pushdown=yes may denote predicate movement inside an engine, partition pruning, connector-side predicate delegation, aggregation delegation, join delegation, limit pushdown, Top-N delegation, or runtime filtering. These actions affect different operators and have different guards, placements, decision times, and observable plan effects. Runtime and adaptivity distinctions are visible in Tukwila’s adaptive integration [39], dynamic query plans [21], and Spark SQL adaptive execution [6]. Rewrite and execution-layer distinctions appear in outerjoin rewrites [29], compiled execution [53], Calcite’s adapter-centered optimizer framework [8], Orca’s modular optimizer architecture [62], and Dremel-style serving trees [50]. A single feature name can therefore erase exactly the fields a comparison needs.

Operator disappearance. A missing native plan node is not a reliable semantics. It may indicate source delegation, logical-plan observability, rewrite, stage-level aggregation, or runtime-only visibility. OptSem-C represents source-delegated execution as a positive contract action rather than as absence. This matters for modern analytical systems whose public surfaces combine Snowflake-style cloud-service stages [23], Redshift’s managed warehouse architecture [35], embedded DuckDB execution [57], vectorized MonetDB/X100 execution [10], distributed SQL services such as F1 [61], and dataflow-style execution such as Spark RDDs [69].

Architecture ranking. A local engine should not be ranked below a distributed engine merely because broadcast, shuffle, or remote-source delegation is not applicable. Conversely, a federated or cloud engine should not be credited with local optimizer behavior when the public contract exposes delegation or service-side stages. OptSem-C separates non-applicability, documented absence, documented optional behavior, and unspecified evidence.

Observable contract versus hidden implementation. The object of study is not what an engine could do internally. It is what the public contract supports as a comparison claim. This distinction is analogous to the separation between SQL equivalence and optimizer-contract equivalence. Cosette proves SQL equivalences by reducing query semantics to logic [19]; semiring-based decision procedures give another equivalence foundation [18]; and QED expands practical SQL equivalence checking [66]. Chandra and Merlin characterize conjunctive-query containment [13]; Aho et al. characterize equivalence among relational expressions [5]. OptSem-C asks a different question: whether the public optimizer behavior being compared exposes the same evidence layers and comparison obligations.

These precision losses imply a minimum payload for comparing public optimizer behavior: guard, canonical action, modality, placement, decision time, observability, version, and provenance. The rest of the paper formalizes this payload and measures what happens when a comparison vocabulary projects it away.

3 CONTRACT FORMALIZATION AND NOTATION

A query probe is a SQL skeleton paired with a finite feature vector and validity constraints. Features describe optimizer-relevant structure: join shape and type, predicate class, aggregation, order/limit behavior, source boundary, connector capability, statistics need, reuse structure, distribution trigger, adaptivity trigger, and optimizer control surface. They are not workload-frequency estimates; they are coordinates on the public optimizer decision surface.

The running P0009 witness is a useful concrete instance before the notation. Under a coarse vocabulary, BigQuery and Trino can both appear as pushdown+explain. The contract payload keeps the distinctions that the label drops: BigQuery federated SQL records guarded source-boundary filter delegation and service-stage evidence, while Trino records connector pushdown and runtime dynamic filtering under different placement, layer, decision-time, and observability fields. The model below exists to preserve exactly those public distinctions without asserting a private optimizer trace.

DEFINITION 1 (PUBLIC CONTRACT RULE). *Let $\mathcal{A}$ be the finite domain of canonical optimizer actions. An action records operator*

Table 1: Conflict-aware state join for contract support states. U denotes UNSPEC, N denotes MUST_NOT, and C marks evidence combinations that require corpus repair before admission.

$\sqcup$	U	MAY	MUST	N
U	U	MAY	MUST	N
MAY	MAY	MAY	MUST	C
MUST	MUST	MUST	MUST	C
N	N	C	C	N

class, action kind, optimizer layer, execution placement, decision time, observability, and variant. The state set $\mathbb{S}$ contains MUST, MAY, MUST_NOT, and UNSPEC. A public contract rule is $r = (\gamma, a, s, v, \epsilon)$, where γ is a guard over query features, $a \in \mathcal{A}$, $s \in \mathbb{S}$, v is the version or time window, and ϵ is evidence provenance. For engine e , C_e is the finite rule set, and $C_e(q) = \{r \in C_e \mid \gamma_r(\phi(q)) = \text{true}\}$ is the rule set applicable to probe q .

The state domain has two components that feature matrices usually conflate. Behaviorally, MUST requires an action, MUST_NOT forbids it, and both MAY and UNSPEC leave it possible. Evidentially, MAY is public support for optional behavior, while UNSPEC is lack of public support. Equivalently, a state is interpreted as (m, u, e) : required behavior, allowed behavior, and evidence support. This is why documented optional pushdown and undocumented implementation possibility are different public contracts.

For each engine-probe pair, OptSem-C constructs a contract map $M_{e,q} : \mathcal{A} \rightarrow \mathbb{S}$, defaulting every action to UNSPEC. Applicable rules update the map with a conflict-aware state join $\sqcup$. The join strengthens unspecified actions when public evidence is found, preserves compatible stronger evidence, and reports contradictions instead of averaging them. The evidence component follows database-provenance practice: semiring provenance attaches support to derived facts [34], why/where provenance distinguishes explanation locations [11], provenance surveys separate data from explanation [16], and annotated containment keeps evidence annotations visible in reasoning [33]. Figure 2 summarizes how these objects become an audit: source-backed rules are admitted, reference maps are materialized for probes, projected equality is tested against reference inequality, and repair certificates record the retained fields. Table 1 gives the local algebra used inside each map.

DEFINITION 2 (STATE-JOIN ALGEBRA). The binary operator $\sqcup$ is the complete 4×4 partial join shown in Table 1. It is total over non-conflicting pairs and returns CONFLICT for pairs that simultaneously require and forbid, or support and forbid, the same canonical action. Let $x \leq y$ denote that $x \sqcup y = y$ for non-conflicting states. Then UNSPEC is the least informative state, while MUST and MUST_NOT are incomparable incompatible commitments.

Here CONFLICT is a merge-failure signal, not a fifth valid contract state; an admitted corpus must resolve it by revising the conflicting evidence record.

LEMMA 1 (MERGE INVARIANTS). On non-conflicting states, $\sqcup$ is commutative, associative, idempotent, and monotone with respect to

$\leq$. Therefore a contract map obtained by joining applicable rules is independent of rule order.

PROOF. The properties follow by inspection of the finite table. Idempotence holds because every diagonal entry is the input state. Commutativity holds because the table is symmetric. Associativity and monotonicity are checked over the finite non-conflicting triples; the only excluded triples are exactly those containing incompatible positive and negative commitments. Lifting the pointwise operator to maps preserves the properties for every action independently. $\square$

Let $W \subseteq \mathcal{A}$ be an abstract optimizer-behavior world. A map M denotes the world set $\llbracket M \rrbracket = \{W \mid M(a) = \text{MUST} \Rightarrow a \in W, M(a) = \text{MUST_NOT} \Rightarrow a \notin W\}$. The evidential support set is $\text{Supp}(M) = \{a \in \mathcal{A} \mid M(a) \in \{\text{MUST}, \text{MAY}, \text{MUST_NOT}\}\}$, equivalently the actions where $M(a) \neq \text{UNSPEC}$. Since maps default to UNSPEC, every supported non-default entry arises from an admitted public rule. Behavioral containment and evidential support are intentionally separate: two maps can allow the same worlds while differing on whether an action is publicly documented.

DEFINITION 3 (REFERENCE SIGNATURES, KERNELS, AND PROJECTIONS). The reference evidence-supported signature of M is $S(M) = \{(a, M(a)) \mid a \in \text{Supp}(M)\}$. Reference equivalence is equality of signatures over the full public evidence atom schema. This reference is exact only relative to the admitted public evidence and declared atom fields; it is not a complete private optimizer trace. A projection is a declared map $\pi : \mathcal{A} \times \mathbb{S} \rightarrow \mathcal{B}$ from full evidence atoms to an arbitrary comparison vocabulary $\mathcal{B}$, such as keyword, yes/no, or operator-only labels. The projected signature is the finite image $S_\pi(M) = \{\pi(a, M(a)) \mid a \in \text{Supp}(M)\}$. The projection kernel is the equivalence relation $\mathcal{K}_\pi$ where $M_i \mathcal{K}_\pi M_j$ iff $S_\pi(M_i) = S_\pi(M_j)$. A projection-induced contract collision is a pair in $\mathcal{K}_\pi$ whose reference signatures differ.

For quantitative comparison, exact compatibility is the Jaccard overlap $|S(M_i) \cap S(M_j)| / |S(M_i) \cup S(M_j)|$, and projected compatibility applies the same formula to S_π . The denominator is evidence-supported: undocumented actions do not count as evidence of either equality or inequality.

LEMMA 2 (PUBLIC-CONTRACT SEPARATION). If $S(M_i) \neq S(M_j)$, then the maps differ in public-contract denotation: either $\llbracket M_i \rrbracket \neq \llbracket M_j \rrbracket$, or their evidence-supported action states differ for at least one action in $\text{Supp}(M_i) \cup \text{Supp}(M_j)$.

PROOF. If reference signatures differ, some supported action has a state in one map that is absent or different in the other. If the difference changes a MUST or MUST_NOT requirement, the world-set constraints differ. Otherwise the behavioral world set may remain equal, but the public evidence state differs, for example MAY versus UNSPEC. In both cases the denotation differs because an OptSem-C contract is the pair of behavioral worlds and evidence-supported states. $\square$

For finite feature and action domains, constructing $M_{e,q}$ from finite C_e and q is a direct finite evaluation: each guard is a Boolean formula over the probe feature vector, and each applicable rule performs one lookup in the state-join table.

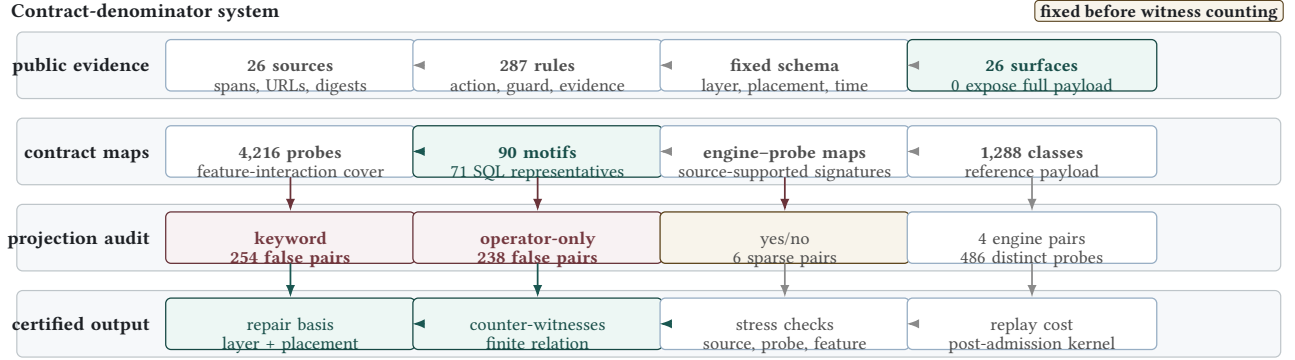


Figure 2: OptSem-C as a denominator-audit system. Public evidence admits rules into a fixed schema; generated probes materialize source-supported contract maps; predeclared projection vocabularies are audited in parallel against reference signatures; and repair certificates, stress checks, and replay costs report the finite comparison boundary.

OBSERVATION (PROJECTION-KERNEL COLLISION). For any projection π , a projection-induced contract collision is exactly the nontrivial part of the kernel $\mathcal{K}_\pi$: M_i and M_j collide iff $S(M_i) \neq S(M_j)$ and $M_i \mathcal{K}_\pi M_j$. If π is non-injective on evidence atoms, there exist contracts with such a collision witness.

PROOF. The first statement unfolds the definitions: $\mathcal{K}_\pi$ is equality after projection, while exact non-equivalence is $S(M_i) \neq S(M_j)$. For the non-injective case, take distinct evidence atoms $(a_1, s_1) \neq (a_2, s_2)$ with $\pi(a_1, s_1) = \pi(a_2, s_2)$. Construct M_1 so that $S(M_1) = \{(a_1, s_1)\}$ and M_2 so that $S(M_2) = \{(a_2, s_2)\}$, with all other actions UNSPEC. These minimal contracts are well formed because action states are assigned independently. Then $S(M_1) \neq S(M_2)$, but $S_\pi(M_1) = S_\pi(M_2)$ is the same singleton, so (M_1, M_2) is a collision witness. $\square$

DEFINITION 4 (PROJECTION INFORMATION PROFILE). For a finite set of observed maps Y , let $H(S)$ be the empirical Shannon entropy of reference signatures and $H(S_\pi)$ the entropy of projected signatures. The entropy-retention ratio $\rho_\pi = H(S_\pi)/H(S)$ measures distinguishability retained by the comparison vocabulary. The projected collision count is the number of projected-signature classes containing more than one reference signature class.

LEMMA 3 (KERNEL INFLATION). If a projection declares E_π equivalent pairs and reference comparison declares E over the same pair set, then E_π/E is the pairwise inflation factor induced by the kernel. Every collision witness contributes to this inflation, and a zero-collision strengthened projection must have $E_\pi = E$.

PROOF. Reference equivalence implies projected equivalence only when the projection is a function of the full atom payload, so $E_\pi \geq E$. The additional pairs counted by E_π are exactly the unequal reference signatures that collapse under π , which are collision witnesses. If there are none, no additional pairs are counted and the two denominators agree. $\square$

OBSERVATION (EVIDENCE REFINEMENT). Adding a non-conflicting evidence-backed rule cannot silently weaken a public contract: it

behaviorally refines possible worlds, evidentially refines support, or both.

The remaining notation is the payoff of the model: it turns an informal vocabulary dispute into a finite certificate check. Let F be the semantic-field universe and let π_R append field values in $R \subseteq F$ to the baseline projection. For a finite comparison set X , define $X_{\pi_R} = \{(M_i, M_j) \in X \mid S(M_i) \neq S(M_j) \wedge M_i \mathcal{K}_{\pi_R} M_j\}$. A repair R is safe for π on X when $X_{\pi_R} = \emptyset$; otherwise any pair in X_{π_R} is a counter-witness. Since adding retained fields can split projected classes but cannot merge already separated classes, safe sets are upward closed and unsafe sets are downward closed. For each witness w , the separator family $\mathcal{D}_w = \{R \subseteq F \mid M_i \mathcal{K}_{\pi_R} M_j\}$ is upward closed, so universal repair is the finite intersection problem $R \in \bigcap_{w \in X_\pi} \mathcal{D}_w$. The reported certificates therefore consist of all repaired witnesses plus counter-witnesses for smaller field sets; exhaustive enumeration over $2^{|F|}$ retained-field subsets is complete for the declared field universe.

PROPOSITION 1 (MINIMUM REPAIR AS HITTING SET). For finite semantic fields F and collision witnesses X_π , deciding whether there is a universal repair of size at most k is NP-complete in $|F| + |X_\pi|$. For OptSem-C's fixed field universe, minimum repairs are exactly enumerable.

PROOF. Membership is finite checking. Hardness reduces HITTING SET: create one witness per set $H_i \subseteq F$ whose two contracts differ exactly on H_i ; a repair separates that witness iff it intersects H_i . $\square$

The semantics makes the comparison payload explicit. A user comparing only operator names chooses a projection that drops placement, decision time, variant, and observability. A feature checklist chooses an even coarser projection that can drop operator class itself. OptSem-C does not forbid those projections; it computes exactly which kernel collisions they induce, which grounded counter-witnesses remain, and which query-processing fields repair the comparison.

4 BENCHMARK AND GROUNDED CORPUS

OptSemBench-C is a behavior-contract benchmark: it generates probes that cover optimizer-relevant feature interactions and measures which public contracts those probes activate. It deliberately does not rank engines by latency. This choice follows the benchmark principle that the metric and denominator must match the object of study. CardEst asks whether cardinality estimates improve DBMS plan choice [36]; CardBench stresses learned cardinality estimators in relational settings [17]; Learned Cardinalities models correlated joins [43]; NeuroCard models probabilistic cardinalities [68]; DeepDB learns database distributions [37]; FactorJoin specializes join-cardinality estimation [67]; Neo learns plan choices [48]; and Bao evaluates learned hinting behavior [47]. An optimizer-contract benchmark must instead expose semantic traps in the comparison vocabulary.

The contract schema is fixed before source extraction, witness counting, and repair search. Its fields come from prior optimizer architecture rather than from the measured collisions. Operator/action and layer follow System R [60], Volcano [30], Cascades [31], Calcite [8], and Orca [62]. Placement follows Garlic federation [12], BigDAWG polystores [27], Dremel serving trees [50], and Snowflake cloud execution [23]. Decision time follows progressive optimization [49], mid-query reoptimization [41], adaptive query processing [26], eddies [7], and Spark adaptive execution [6]. Observability is admitted from plan, profile, and stage evidence in the public source ledger, with provenance-style evidence separation [34]. Layer and placement enter the predeclared field universe; exhaustive retained-field testing later reports them as a sufficient headline repair basis, not as fields selected after seeing failures.

Let $F = F_1 \times \dots \times F_n$ be the finite feature domain and Ψ the validity constraints. A target interaction is a valid partial assignment over a feature subset. The schema and feature domain are fixed before witness counting and derive from standard query-processing constructs and cited public optimizer surfaces, not from witnesses or motifs. The benchmark targets global pairwise coverage plus selected three-way interactions for source boundary, connector capability, operator class, statistics need, join shape, adaptivity trigger, and distribution trigger. The generator builds one deterministic candidate pool from renderable feature interactions and admitted public-rule guards. It first includes 99 guard-forced completions so that every admitted rule has a reachable query shape, then greedily covers the remaining feature-interaction universe. These two roles are reported separately: guard-forced probes prevent public contracts from becoming unreachable paper claims, while the remaining probes complete the declared feature-interaction denominator. Neither role is treated as an independent held-out population sample. A probe is useful when it activates a contract guard or separates two contract signatures, not when it estimates workload frequency.

OBSERVATION (COVERAGE). *If every valid target interaction has at least one renderable full assignment, the generator terminates with a probe set covering every renderable target interaction.*

PROOF. The target universe is finite. Each selected assignment covers at least one uncovered interaction. Thus the number of

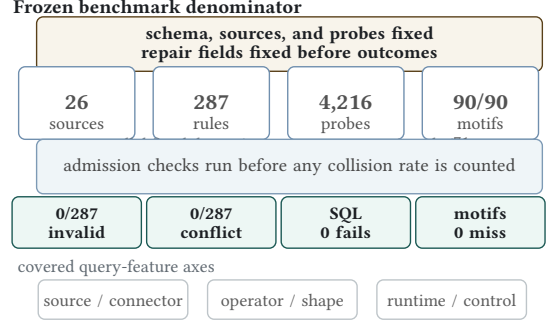


Figure 3: Corpus, benchmark, and execution denominators: 26 public sources, 287 admitted rules, 4,216 executable probes, and 90/90 published motif requirements represented by 71 distinct generated SQL probes. These are parallel fixed denominators; zero-failure checks validate support, consistency, SQL execution, and motif coverage before collision measurement.

uncovered interactions decreases monotonically until it reaches zero. $\square$

The main empirical corpus contains only 287 source-linked grounded rules. Each rule records a public evidence span, source URL, line range, digest, and source class. Claims without grounded evidence, expressible guards, or public observability are outside the finite denominator rather than negative evidence. Thus the paper claims coverage of the declared public-contract denominator, not a census of private optimizer rules. Figure 3 gives the denominator view before any collision measurement: sources, rules, probes, and motif requirements are fixed, and the zero-failure checks verify support, consistency, SQL execution, and motif coverage. The corpus covers plan observability, pushdown, join search, distribution, materialization, runtime adaptivity, statistics, and optimizer control surfaces. Every grounded rule is triggered by at least one generated probe, and contract merge produces no conflicts.

The source set is fixed before witness counting. We admit an engine only when public sources provide: a comparable plan or contract surface, enough detail to extract an action and applicability guard, and stable analytical-SQL behavior rather than a private vendor note. PostgreSQL, Trino, BigQuery, Spark SQL, Snowflake, DuckDB, and ClickHouse enter through versioned source records in the evidence ledger; Calcite and DataFusion source records provide schema context rather than denominator engines. These records are not scholarly support and not performance measurements. They are the measured public contracts that the framework formalizes. Proprietary workload logs, undocumented cost-model traces, future engine versions, and operator families outside the declared feature domain are excluded rather than silently counted.

The source-linked denominator spans seven engines: PostgreSQL contributes 80 admitted rules, Trino 61, BigQuery 37, Spark SQL 36, ClickHouse and Snowflake 26 each, and DuckDB 21; the largest single source supports 46 rules. This design separates admission, probing, and execution validation. Public sources admit rules; generated probes test guard reachability and SQL executability; DuckDB

Table 2: Source-to-rule traceability for representative contracts. Each row binds a rule state/action to a public source span, digest prefix, and paraphrase used by the replay manifest.

Contract rule			Auditable evidence	
Engine	State	Action	Span/digest	Source-supported paraphrase
PostgreSQL	MAY	Plan/observe/local	PG-EXPLAIN-001 bfb47e66 L46-L56	EXPLAIN exposes estimated startup and total cost, output rows, row width, and planner-cost units.
BigQuery	MAY	Exchange/adapt/cloud	BIGQUERY-PLAN-001 cc7b206a L990-L992	The query plan may be modified during execution by adding repartition stages.
Snowflake	MAY	Plan/observe/cloud	SNOWFLAKE-EXPLAIN-001 93a61359 L150-L154	EXPLAIN compiles a statement and returns a logical execution plan rather than executing it.
ClickHouse	MAY	Join/adapt/local	CLICKHOUSE-JOINING-001 a1f757e2 L313	With automatic join-algorithm selection, execution can switch from hash join to partial merge join.
DuckDB	MAY	Materialize/inline/local	DUCK-WITH-001 2a60c8ab L482-L487	CTE inlining is guarded by single use, absence of volatility, no grouped aggregation, and inlining hints.
Spark SQL	MAY	Plan/adapt/distributed	SPARK-PERF-001 7e5e2b00 L129-L133	Adaptive query execution uses runtime statistics and can re-optimize a query while it is running.
Trino	MUST_NOT	Agg./pushdown/source	TRINO-PUSH-001 cc177219 L168-L176	Aggregation pushdown does not support aggregate calls containing expressions.

and PostgreSQL runs validate the probe denominator on two open engines, not every vendor contract. All-engine execution would conflate SQL validity with public-contract admission. A DuckDB source span can admit a DuckDB public contract, but a later DuckDB execution is only SQL execution and visible-plan sanity evidence; it never admits rules or chooses a repair field. Conversely, engines without local execution still participate in collision measurements only through source-linked public contracts, source leave-one-out checks, and side-balanced witness support.

The grounded corpus is intentionally conservative. A rule is admitted only when the evidence span supports an optimizer action, a guard or applicability condition, and an observability or placement interpretation. Vague performance advice is excluded from the main corpus. This policy reduces corpus size but strengthens the central claim: the evaluation is about contracts that are publicly supported, not about inferred implementation behavior.

Contract extraction and validation checks. Each admitted rule records the engine, version scope, source identifier, digest, action fields, guard, and observability surface. Human coding maps public evidence spans into the fixed schema; automated checks validate schema consistency, source linkage, guard reachability, trigger coverage, merge conflicts, SQL executability, and table-source consistency. This is a single-extractor source-line protocol rather than formal verification or recoding agreement: every accepted row is inspectable, vague performance advice is excluded by rule, and excluded rows are outside the declared denominator. Table 2 gives representative spans; the replay materials carry the complete rule-to-source manifest, source digests, claim-evidence graph, and regenerated table manifests. The field schema and projection portfolio are replay inputs fixed before collision counting. Layer plus placement is reported as an interpretable repair basis for headline witnesses after enumerating retained-field vocabularies; the frontier tables give scoped safety and do not claim global safety. Public source records admit claims; EXPLAIN, profile, or stage surfaces bind exposed plan evidence when available; SQL execution validates the probe denominator rather than private optimizer truth. A counted witness requires two source-supported reference signatures, projection equality under the declared vocabulary, and separation by the reported repair field.

The denominator dashboard in Table 3 checks that this conservatism does not create unreachable contracts or syntactic probes. A finite-disjunction guard semantics evaluates each admitted rule against the generated probe denominator. All 287 grounded rules have at least one triggering probe; 252 depend on query features rather than global engine labels; and no guard uses a feature dimension or value outside the benchmark domain.

Coverage is reported over two denominators. The first is benchmark coverage: every targeted renderable valid feature interaction is covered by at least one generated probe. The second is contract triggering: every grounded rule is applicable to at least one probe. Together these denominators make the empirical scope explicit. A rule that cannot be triggered by any probe would indicate a benchmark gap; a feature interaction that cannot be rendered is excluded from the coverage denominator and reported as a modeling constraint.

The benchmark also exposes executable SQL shapes. Each generated probe has a normalized SQL skeleton and shape checks for joins, filters, grouping, ordering, limits, and reuse structure. The full 4,216-probe corpus is accompanied by a compact 48-probe motif-cover prefix. The real-engine motif execution set uses the deterministic union of this motif cover and residual feature-axis maxima; it is not a hand-picked success set, and it is not a replacement for the full evaluation.

A common failure mode in generated benchmarks is to produce feature-complete but non-executable query text. OptSemBench-C therefore validates every SQL skeleton on a deterministic relational catalog with local, remote, and cross-source namespaces. This validation is not a performance baseline and is not exhaustive contract-trigger proof. Its role is stricter and more basic: every generated probe must be parseable, plannable, executable, and shape-consistent with its feature vector. Table 3 reports the validation result: all 4,216 generated probes obtain a plan and execute, with zero SQL-shape failures. Repeating the full corpus across three deterministic catalog populations yields 12,648 successful probe-catalog executions, and DuckDB/PostgreSQL checks add 8,432 full-corpus engine-probe executions plus 142 motif-representative executions with zero execution failures.

Table 3: Benchmark denominator and validation checks across corpus, SQL, real-engine SQL execution, and motif layers. Bold marks complete coverage or zero-failure checks; execution rows validate query construction, not contract truth.

Layer	Check	Value	Scope	Interpretation
Corpus	grounded rules	287	26 sources	finite public-contract relation
Corpus	probe-supported rules	287/287	median support 102	every admitted rule is triggerable
Corpus	non-global guards	252	3 containments	query-conditional, audited overlaps
Guard schema	invalid dimensions	0	fixed feature domain	no out-of-domain guard values
SQL corpus	generated probes	4,216	digest df107f018e37	full executable denominator
SQL corpus	motif-cover prefix	48	human inspection	compact replay slice, not cherry-picking
SQL execution	planned/executed probes	4,216/4,216	91 distinct plans	every generated probe plans and runs
SQL execution	shape/exec failures	0/0	18,448 rows	no SQL-shape or runtime failures
Real engines	full DuckDB/PostgreSQL	8,432/8,432	tags .705/.796	SQL validation, not source truth
Real engines	motif DuckDB/PostgreSQL	142/142	71 reps $\times$ 2	budgeted open-engine sanity
Published motifs	covered requirements	90/90	71 representatives	shared probes are explicit, not hidden

Table 4: Probe budgets for triggering grounded contract rules; random columns summarize subset baselines. Bold marks complete trigger coverage.

Budget	Cover	Diagnostic	Random mean	Random 5–95
50	.672	.997	.707	[.627,.767]
100	1.000	1.000	.811	[.770,.850]
250	1.000	1.000	.890	[.864,.913]
500	1.000	1.000	.920	[.899,.941]
1000	1.000	1.000	.951	[.934,.969]
2000	1.000	1.000	.973	[.958,.986]
4216	1.000	1.000	1.000	[1.000,1.000]

Contract validation is layered. Guard reachability says that each public rule has a generated probe satisfying its admitted guard; SQL execution says that the denominator is not syntactic fiction. Visible-plan sanity checks on DuckDB and PostgreSQL expose expected plan tokens at mean rates 0.705 and 0.796 over the full corpus, and 0.756 and 0.819 on motif representatives. These checks do not reverse-engineer private optimizers, prove every documented behavior fires on every cost-model instance, or cross-validate a source by running the same engine. A counted witness still requires source-supported reference signatures, guard reachability, projection equality, and repair separation; runtime plan satisfaction is a separate SQL-denominator validation layer.

The same fixed probes support two budgeted evaluation orders. The cover order is the generator’s feature-interaction order. The diagnostic order greedily maximizes newly triggered grounded rules without adding any new probe; it is a rule-coverage schedule over admitted contracts, not an unseen workload sample. Table 4 reports both. The cover order triggers all grounded rules within 100 probes; the diagnostic order triggers 286 of 287 rules within 50 probes and all rules within 100. Random subsets of 100 probes trigger 81% of rules on average and 85% at the 95th percentile. Thus OptSemBench-C is not merely large enough to cover the surface; it also contains compact diagnostic prefixes that expose almost all grounded contracts.

A benchmark for public optimizer contracts must connect to existing database workloads without becoming another latency benchmark. We therefore crosswalk published optimizer and benchmark motifs to OptSemBench-C feature requirements: JOB join-order sensitivity [45], POLAR-style join structure [40], TPC decision-support

Table 5: Matching-probe difficulty across published optimizer motifs. Underlining marks the sparsest and broadest motif coverage extremes; motif-family sources are cited in the crosswalk paragraph.

Suite	Motifs	Min	Median	Max
TPC-H	8	<u>1</u>	87.0	1384
TPC-DS	8	86	164.5	287
JOB	8	55	87.0	236
SSB	6	89	101.0	<u>3442</u>
CardEst	7	72	228.0	259
Adaptive	6	72	205.5	235
Federated	8	72	91.5	633
Controls	8	70	72.0	73
PostgreSQL	8	72	73.0	73
DuckDB	8	68	72.0	364
ClickHouse	7	70	70.0	70
Spark/Trino	8	69	70.0	236

benchmarking [56], TPC-DS design [52], SSB [54], LDBC [28], Wisconsin [9], warehouse surveys [24], YCSB [22], and LakeBench [25]. Figure 4 reports the full executable denominator: 90 declared requirements across 12 motif families, represented by 71 distinct generated SQL probes because several queries satisfy more than one optimizer-decision requirement. Five representative anchors illustrate the span: JOB join-order search (8/8 requirements), exact-cardinality testing (9/9), CardEst-style plan-quality sensitivity (8/8), adaptive replanning (9/9), and cross-source joinability (7/7). Table 5 gives the 12-family difficulty distribution. The selected probes are feature-interaction representatives, not replacements for JOB or TPC-H queries; the crosswalk validates optimizer-decision coverage rather than workload runtime behavior. It remains a frozen audit over optimizer-decision surfaces, not copied benchmark SQL or a latency claim; motifs outside the declared feature domain would be denominator misses. Because motifs are checked after the schema is fixed and before repair fields are used, this is a non-circular denominator check: a counted collision still requires two source-supported signatures, projection equality, side-balanced support, and repair separation. We claim contract-denominator adequacy and motif representation, not workload reproduction.

The motif-difficulty view prevents a misleading denominator. Some motifs are broad and match many probes, such as star-schema group-by shapes. Others are sparse, such as a TPC-H conjunctive-filter motif that has one matching rendered probe under the current validity constraints. Reporting both coverage and matching-probe

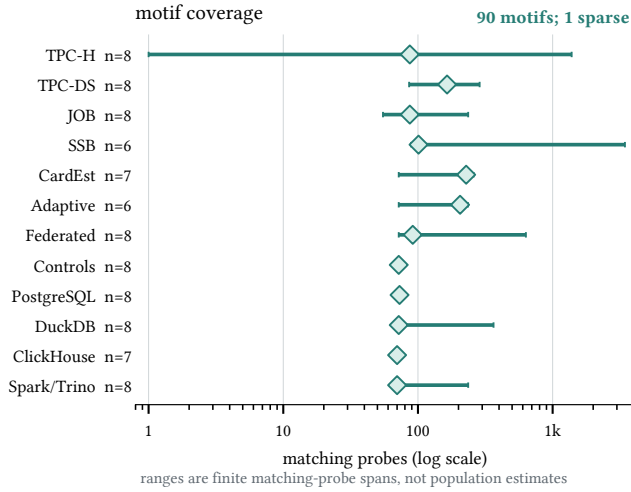


Figure 4: Published optimizer-motif denominator from the frozen table of 12 motif families and 90 requirements. These requirements validate through 71 distinct SQL representatives; a matching-probe span is the minimum–maximum number of OptSem-C probes that trigger a family, so sparse motifs stay visible.

counts makes benchmark adequacy inspectable: coverage says the motif is present, while difficulty says how concentrated or redundant that coverage is. The executable motif suite adds a second denominator check: for each published motif, the system selects one satisfying probe and validates the generated SQL on the deterministic catalog, yielding 90 validated motif requirements, 71 distinct representatives, and 383 result rows. Thus the motif denominator is not a list of names; it is a set of optimizer-decision requirements with concrete SQL representatives and an explicit many-to-one map. Engine-plan validation is a separate experiment because the motif suite itself is a denominator check rather than a claim that published benchmark queries and generated probes induce identical plans.

5 EVALUATION

The evaluation follows the objection path a comparison paper would face. Do coarse public vocabularies create denominator errors? Are counts grounded by witnesses rather than names? Do source, probe, feature, and engine stresses preserve a finite kernel without becoming population claims? Are projection-audit stages cheap enough to rerun when the denominator changes? The schema, projections, source manifest, and repair-field universe are frozen before counting. Execution validates SQL probes but never admits rules or chooses repair fields; a non-interference check verifies that DuckDB/PostgreSQL validation files are not consumed by collision or repair tables. The table column “false” is contract-level: a declared vocabulary collapses unequal public optimizer-contract signatures before any latency, plan-quality, or cost claim is interpreted.

The comparison surfaces are executable projection vocabularies and controls, not external DBMS-testing algorithms. Across the 26

Table 6: Main projection audit, combining collision rate, witness severity, and repair diagnosis. Mean/Max Δ count differing public atom fields inside collapsed reference signatures. Stars mark headline public vocabularies; bold numeric zeros mark controls or repairs, and underlining marks the largest unsafe stress row.

Projection	Eq.	False↓	Rate↓	Mean Δ	Max Δ	Diag.
Exact	2,030	0	0.0%	0.00	0	ctrl
Keyword*	2,284	254	11.1%	9.09	17	placement
Yes/No	2,036	6	0.3%	3.00	3	layer
Operator-only*	2,268	238	10.5%	5.84	14	layer
Placement-only	12,732	10,702	84.1%	21.29	53	stress
State-only	25,623	23,593	<u>92.1%</u>	13.06	<u>64</u>	stress
Surface	2,030	0	0.0%	0.00	0	repair

public source surfaces, keyword, yes/no, and operator projections mirror feature labels, enablement controls, and operator-family labels; none exposes the complete reference contract payload. Reference comparison is the negative control, one-field projections are diagnostic ablations, and strengthened projections restore fields to localize missing information. All projections are fixed before witness counting and evaluated over the same source-linked maps. The yes/no surface is the binary projection induced by public enablement or control claims; it is intentionally narrow because such controls often omit placement and evidence layers. The question is how many witnesses are lost if equality is decided at that vocabulary, not whether every cited study made that exact projection. Workload controls follow TPC-style analytical benchmarks, join-order diagnostics follow JOB [45] and its retrospective analysis [46], and the plan-quality control follows exact-cardinality testing [15].

Coarse comparison induces repairable collisions. The main empirical result is a finite counterexample to common optimizer-comparison vocabularies. Table 6 keeps public vocabularies, controls, stress rows, and repairs on the same scale: keyword declares 2,284 equivalences and collapses 254 unequal reference signatures, while operator-only declares 2,268 and collapses 238. These errors are conditional on the vocabulary declaring equality over the fixed denominator. They are also concentrated rather than uniform: the 498 headline records occur in four observed engine pairs and 486 distinct probes, with BigQuery–ClickHouse, BigQuery–Trino, ClickHouse–Trino, and Spark SQL–Trino carrying the visible hotspots. This concentration is part of the claim, not an inconvenience hidden by the aggregate count. Headline collisions have reference-signature Jaccard distance 1.0 and differ by 9.09 and 5.84 evidence atoms on average. The strict reference and strengthened surface controls have zero false pairs; one-field stress rows show that a single optimizer dimension can make the denominator worse rather than safer.

5.1 Concrete Collision Audits

Table 7 grounds the first result with four concrete audits from the witness ledger behind Table 6. The cases are deliberately small: each starts from a familiar public label, shows which contracts that label hides, and names the fields that separate the witness. C1 is the running P0009 example: a coarse pushdown/explain label hides

Table 7: Concrete collision audits linking coarse labels to hidden contract pairs. Repair keys are case-local separators; the reported layer+placement basis covers all headline records.

Case	Hidden comparison	Repair read
C1	P0009 links four sources where pushdown+explain hides BigQuery source delegation and stage evidence versus Trino connector pushdown and dynamic filtering.	Operator, placement, and decision time separate the public contracts.
C2	Sixteen estimate and plan-evidence rules mix local estimates, cloud-service stages, and distributed-worker evidence under one exposed-plan label.	Observability and placement distinguish what the audit can inspect.
C3	Sixteen adaptive-execution rules merge compile-time planning with runtime repartition, AQE, and conversion behavior.	Layer and decision time separate when the optimizer commits.
C4	Sixteen CTE rules collapse inline, materialize, reuse, and disable-guard contracts into one materialization label.	Variant and guard state identify the case-local separator.

Table 8: Projection-surface portfolio used for controls, public-label failures, stress surfaces, and repairs. Bold numeric zeros mark safe controls or repairs; underlining marks the worst shown risk row.

Surface	Eq.	False	Rate
Exact	2,030	0	0.0%
Keyword	2,284	254	11.1%
Op/action	2,036	6	0.3%
Operator	2,268	238	10.5%
Kind-only	2,282	252	11.0%
Layer-only	2,479	449	18.1%
Place-only	12,732	10,702	84.1%
State-only	25,623	<u>23,593</u>	<u>92.1%</u>
Surface	2,030	0	0.0%

BigQuery filter delegation, BigQuery stage evidence, Trino connector pushdown, and Trino dynamic filtering. C2–C4 cover estimate observability, runtime adaptation, and CTE materialization; the same finite checker covers all 498 headline witnesses.

Table 8 shows representative rows from the 17 executable projection surfaces: controls, public-label failures, stress tests, repairs, and the finite resolution frontier. These are comparison vocabularies and stress surfaces, not competing DBMS-testing methods. Zero rows are controls or repaired vocabularies; nonzero rows are fixed public projections or retained-field stress tests; repair rows rest on smaller-field counter-witnesses rather than added labels.

Finite resampling keeps keyword at roughly 10–12% and operator labels at roughly 9–12%. Single-source removal is stricter to read: removing one source can delete original witnesses and create new ones, so the audit reports both nonzero counts and witness-identity movement rather than treating leave-one-source as population sampling. The repair column localizes the missing semantics: placement repairs keyword, layer repairs the operator row, and the strengthened semantic surface returns to zero false pairs. The sparse yes/no row remains a narrow witness family.

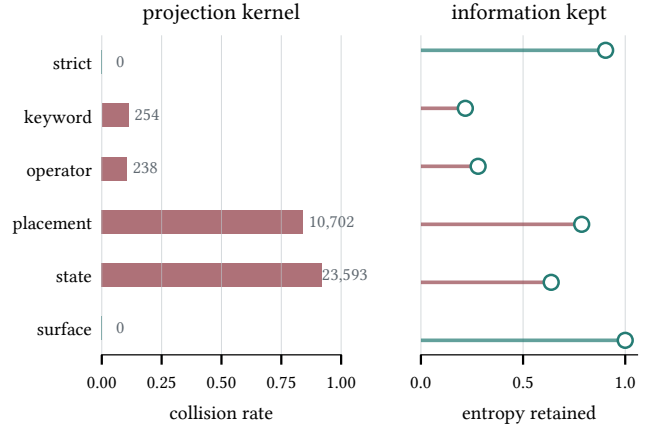


Figure 5: Projection information profile from 24,342 rule-probe maps and 88,536 declared pair comparisons. A projection kernel is the set of reference-unequal contracts collapsed by a vocabulary; labels give false-equivalence counts, and interval checks remain in Table 10.

Projection kernels are measurable objects. Figure 5 reports the finite information profile of representative projections. Reference signatures form 1,288 observed classes. Keyword projection compresses them into 148 projected classes, 81 of which contain multiple reference classes, retaining 64% of empirical signature entropy and inflating declared equivalence by 12.5%. Operator-only projection retains more entropy but still leaves 191 colliding projected classes and 238 pairwise collisions. Placement-only and state-only surfaces demonstrate the opposite failure mode: a single field creates very large kernels. The strengthened surface projection still has projected collisions at the signature-class level, but it does not produce any pairwise collision over the engine-probe comparison set. Thus the kernel is finite, populated, and repairable in the observed comparison relation.

Resolution-lattice stress test. Table 9 asks which retained-field vocabularies are safe when a comparison may keep any subset of public optimizer fields. The exhaustive test enumerates all 256 subsets over the same 88,536 engine-probe comparisons. This is not a repeated-significance search: every unsafe cell is a concrete counter-witness in a fixed finite relation, so no multiple-testing correction is being used to select a result. The empty vocabulary declares 59,546 unsupported equivalences, and every singleton semantic field remains unsafe: operator leaves 238 collisions, kind 252, layer 449, and placement 10,702. Excluding action-variant identifiers, the semantic lattice has 128 subsets, 44 unsafe regions, minimum safe field count two, 14 minimal safe vocabularies, and eight maximal unsafe vocabularies. Ten concrete engine-probe witnesses cover all unsafe retained-field regions, so the monotone frontier certificate is finite rather than anecdotal.

The robustness view in Table 10 checks that the headline result is not an effect of a single denominator. The micro rate conditions on the projection declaring equivalence; finite bands and probe-cluster resampling summarize dispersion over declared equivalences and

Table 9: Retained-field lattice for projection repair. Ent. kept is higher-better, Eq. is denominator size, and False is lower-better. All sem. excludes action-variant identifiers, so entropy can remain below 1.0. Bold zeros mark safe vocabularies; underlining marks the largest unsafe collision count.

Fields	R	Classes	Ent. kept↑	Eq.	False↓
None	0	1	0.000	61,576	59,546
Operator	1	387	0.787	2,268	238
Action kind	1	183	0.661	2,282	252
Layer	1	52	0.552	2,479	449
Placement	1	9	0.280	12,732	10,702
Operator+layer	2	591	0.850	2,030	0
Kind+placement	2	214	0.680	2,030	0
Surface	5	820	0.904	2,030	0
All sem.	7	947	0.934	2,030	0

probe clusters, not population inference; the pair-macro rate averages within engine pairs; and the leave-one-source range removes all rules supported by one public source. Keyword and operator-only collisions remain nonzero across these views, with leave-one-source conditional-rate ranges [0.017, 0.144] and [0.032, 0.148] respectively; the Boundary panel records how much of the original witness set is retained and how many new witnesses are induced by deletion.

The evidence side is cross-checked without retraining on headline records: every collision has at least two public source records and side-balanced support. The freeze manifest records the schema, feature domain, source manifest, admitted rules, projection portfolio, motif crosswalk, result-determining implementation, and replay entrypoints before witness counting; changing them requires a full replay.

The Boundary panel in Table 10 is a ledger rather than a victory table. Keyword and operator-only remain nonzero after every leave-one-source deletion, but keyword retains as little as 2% of its original witness identities in the most disruptive deletion and can induce many new witnesses. The point is therefore not that source removal preserves the same sample; it is that a nonzero kernel remains while the identity movement is exposed. Their 10% probe subsamples remain nonzero in all trials, so the kernel is not produced by one source or probe slice inside the frozen denominator. The yes/no projection is source-sensitive and stays out of the broad robustness claim. The reported layer-and-placement basis resolves feature-family and engine-family stress, while point-learned engine-pair repairs fail on held-out pairs; that failure keeps the claim finite-denominator rather than learned-transfer.

Because the 287-rule corpus is finite, the remaining checks test whether the kernel is a one-source or one-shape accident. The seven-engine denominator contributes admission evidence, controls, and source-removal stress; the 498 headline false-witness records occupy four observed engine pairs and 486 distinct probes. They touch all 12 feature dimensions and fully cover the value domain for 10 of them, while every probe carries at most one headline witness. The dispersion matters because the result is concentrated in interpretable engine pairs without collapsing to a single probe or feature value.

Finite-audit replay cost. A VLDB evaluation should not have to trust an expensive hand audit for the comparison kernel. The

core state-free repair frontier

	retained depth			basis	safe/unsafe
	0	1	2+		
keyword (254)	U	S	-	place (4)	23/9
yes/no (6)	U	S	-	place (2)	24/8
operator (238)	U	S	-	layer (3)	22/10
all (498)	U	U	S	layer + place (6)	21/11

Figure 6: Scoped repair frontier from the core semantic, state-free retained-field enumeration: four projection scopes by three retained-field depths. U=unsafe, S=first safe repair, dash=safe non-minimal, and parenthesized basis values count minimum repair bases.

Replay panel in Table 10 reports projection audit, SQL validation, paper-facing replay total, and an 8× lift over the same constructed relation. The measurements are post-admission projection-kernel replay costs; they exclude setup, paper build, package construction, source acquisition, and fresh DBMS reruns. The same panel adds projection-level dispersion: full audits cover 88,536 engine-probe checks per projection above 50,000 checks/s, the 8× lift repeats the relation to 708,288 checks per projection above 190,000 checks/s with prefix-scaling $R^2 \geq 0.98$, streaming maintenance has zero drift, and the largest source refresh affects 46 rules, 4,216 maps, and 14.3% of rule-probe rows.

Repair certificates identify the missing semantics. Table 6 reports collision counts, severity, and minimum repair diagnosis. The certificate evidence is stricter than a post-hoc label choice: each certificate must match the tested projection, differ under reference contracts, separate under the proposed repair, and survive counter-witness checks against smaller universal repairs. Placement repairs keyword collisions; layer repairs operator-only collisions; layer plus placement repairs all 498 headline records without action-variant labels. Residual paths show why one field is insufficient: operator nearly repairs keyword but leaves six collisions, and placement alone still fails for the operator row. Fold-local repairs can underfit the global denominator; layer+placement resolves every probe-fold collision. Figure 6 gives the scoped repair frontier: safe subsets repair in-scope collisions, unsafe subsets retain counter-witnesses, and layer+placement is a headline-witness basis rather than a globally safe vocabulary.

Collisions have mechanisms and hotspots. The repair fields are query-processing boundaries: placement separates local execution, source delegation, distributed workers, and cloud services; layer and decision time separate rewrite, planning, distribution, and runtime adaptation; operator and observability keep estimates, plans, and profiles distinct. The finite witness ledger attributes all 498 headline records to action variants, 466 to placement, 430 to optimizer layer, 366 to plan evidence, 362 to decision time, and

Table 10: Freeze, robustness, boundary, and replay dashboard. Down arrows mark lower-better risk or cost; bold zeros are controls or repaired surfaces. All rates are finite-denominator summaries, not population estimates.

Panel	Row	False↓	Rate/Cost	Stress	Boundary read
Freeze	Denominator	–	pre-count	SHA manifest	schema, sources, projections, motifs, and replay code locked
Collision	Strict reference	0	0.000	control	full public payload, no induced equality error
Collision	Keyword labels	254	0.111	finite [.099,.125]	public keyword cells hide placement distinctions
Collision	Operator labels	238	0.105	finite [.093,.118]	operator-family equality hides optimizer layer
Collision	Strengthened surface	0	0.000	repaired	retained fields remove headline witnesses
Boundary	Source LOO	–	nonzero 26/26	kept/new tracked	information deletion, not exchangeable sampling
Boundary	Pair + basis	0	498/498 repaired	4 pairs; 486 probes	concentrated signal with scoped layer+placement repair
Replay	Replay total	–	3.50 s	455,828 input rows	excludes source admission and package construction
Replay	8× audit lift	–	2.49 s	708,288 checks/projection	deterministic inner-loop lift, not new engines

281 to operator family. The hotspots are also interpretable: keyword collisions are concentrated in BigQuery–ClickHouse and BigQuery–Trino comparisons, while operator collisions concentrate in ClickHouse–Trino and Spark SQL–Trino.

Workload sensitivity. Controlled feature-toggle movements over CTE reuse, source boundary, statistics need, distribution, and adaptivity show that comparability is a relation between engine contracts and query structure. The largest movements are finite neighborhoods over the frozen feature space rather than broad performance claims: DuckDB reuse structure has the largest mean signature movement, Trino statistics need has broad nonzero support, and distribution/adaptivity toggles move different engines through different public-contract coordinates.

6 ARCHIVE AND REPLAY

Each paper-facing number is bound to a replay ledger over a fixed schema, source set, projection vocabulary, motif crosswalk, probe denominator, and repair universe; changing any input creates a new denominator and requires a full replay. The archive keeps the 70-paper bibliography separate from source-record ledgers. DuckDB and PostgreSQL executions validate that generated SQL runs over deterministic catalogs, but those outputs are scoped away from collision and repair counts. The timed rows measure only post-admission finite-audit replay; source discovery, manual admission, package construction, and fresh engine installation remain outside the reported costs. They are engineering replay bounds for an already admitted denominator, not human-audit baselines.

7 DISCUSSION AND LIMITATIONS

Use and boundaries. OptSem-C is a pre-ranking audit. Before comparing pushdown, join processing, materialization, adaptivity, or plan evidence, a study should name the action, guard, optimizer layer, execution placement, decision time, and evidence surface being compared. A zero row is not an optimizer-correctness theorem; it only says that the tested public vocabulary did not collapse unequal reference signatures in this finite relation. The work does not

infer private cost models, rank runtime performance, or turn public documentation into a population guarantee. New engines, proprietary traces, undocumented switches, and out-of-schema operators require new admission and replay; source removal is information deletion, so retained and induced witnesses are reported separately.

8 RELATED WORK AND CONCLUSION

Optimizers and analytical systems. System R separates access paths and costing [60]. Volcano supplies rule search [30]; its generator makes search programmable [32]; and Cascades gives the memo model [31]. Calcite [8] and Orca [62] expose framework boundaries. C-Store [64], column-store comparisons [3], compression [2], materialization [4], and a column-store survey [1] show why representation matters. Vectorwise [70], HyPer [42], Hive [65], Impala [44], Garlic [12], BigDAWG [27], Dremel [50], Snowflake [23], Redshift [35], and F1 [61] expose placement or service boundaries.

Benchmarks. TPC decision-support benchmarking [56], TPC-DS [52], SSB [54], Wisconsin [9], and data-warehouse benchmark surveys [24] standardize workload ideas. JOB [45] targets join-order behavior. Pavlo et al. compare large-scale analysis systems [55]; YCSB studies cloud serving [22]; and LakeBench [25] stresses lake discovery. CardEst [36], CardBench [17], Kipf et al.’s learned estimator [43], NeuroCard [68], DeepDB [37], and FactorJoin [67] evaluate cardinality or distribution models. Neo [48] and Bao [47] test learned planning. OptSem-C asks whether public optimizer behaviors grouped under a label are the same public contract.

Testing and evidence. SQLancer [59] and NoREC [58] search for DBMS and optimization bugs. Cosette [19], semiring axioms [18], QED [66], containment [13], and relational equivalence [5] supply SQL verification background. Why/where provenance [11], semiring provenance [34], provenance surveys [16], and annotated containment [33] keep evidence in reasoning.

Conclusion. OptSem-C makes public optimizer labels auditable before runtime rankings.

REFERENCES

- [1] Daniel Abadi, Peter Boncz, Stavros Harizopoulos, Stratos Idreos, and Samuel Madden. 2013. The Design and Implementation of Modern Column-Oriented Database Systems. *Foundations and Trends in Databases* 5, 3 (2013), 197–280. <https://doi.org/10.1561/19000000024>
- [2] Daniel J. Abadi, Samuel R. Madden, and Miguel Ferreira. 2006. Integrating Compression and Execution in Column-Oriented Database Systems. In *SIGMOD*. 671–682. <https://doi.org/10.1145/1142473.1142548>
- [3] Daniel J. Abadi, Samuel R. Madden, and Nabil Hachem. 2008. Column-Stores vs. Row-Stores: How Different Are They Really?. In *SIGMOD*. 967–980. <https://doi.org/10.1145/1376616.1376712>
- [4] Daniel J. Abadi, Daniel S. Myers, David J. DeWitt, and Samuel R. Madden. 2007. Materialization Strategies in a Column-Oriented DBMS. In *ICDE*. 466–475. <https://www.cs.umd.edu/~abadi/papers/abadiicde2007.pdf>
- [5] A. V. Aho, Y. Sagiv, and J. D. Ullman. 1979. Equivalences among Relational Expressions. *SIAM J. Comput.* 8, 2 (1979), 218–246. <https://doi.org/10.1137/0208017>
- [6] Michael Armbrust, Reynold S. Xin, Cheng Lian, Yin Huai, Davies Liu, Joseph K. Bradley, Xiangrui Meng, Tomer Kaftan, Michael J. Franklin, Ali Ghodsi, and Matei Zaharia. 2015. Spark SQL: Relational Data Processing in Spark. In *SIGMOD*. 1383–1394. <https://doi.org/10.1145/2723372.2742797>
- [7] R. Avnur and J. M. Hellerstein. 2000. Eddies: Continuously Adaptive Query Processing. In *SIGMOD*. 261–272. <https://doi.org/10.1145/342009.335420>
- [8] E. Begoli, J. Camacho-Rodriguez, J. Hyde, M. J. Mior, and D. Lemire. 2018. Apache Calcite: A Foundational Framework for Optimized Query Processing Over Heterogeneous Data Sources. In *SIGMOD*. 221–230. <https://doi.org/10.1145/3183713.3190662>
- [9] D. Bittton, D. J. DeWitt, and C. Turbyfill. 1983. Benchmarking Database Systems: A Systematic Approach. In *Vldb*. 8–19. <https://www.vldb.org/conf/1983/P008.PDF>
- [10] P. A. Boncz, M. Zukowski, and N. Nes. 2005. MonetDB/X100: Hyper-Pipelining Query Execution. In *CIDR*. 225–237. <https://www.cidrdb.org/cidr2005/papers/P19.pdf>
- [11] P. Buneman, S. Khanna, and W.-C. Tan. 2001. Why and Where: A Characterization of Data Provenance. In *ICDT*. 316–330. https://doi.org/10.1007/3-540-44503-X_20
- [12] Michael J. Carey, Laura M. Haas, Peter M. Schwarz, Manish Arya, William F. Cody, Ronald Fagin, Myron Flickner, Allen W. Luniewski, Wayne Niblack, Dragutin Petkovic, John Thomas, John H. Williams, and Edward L. Wimmers. 1995. Towards Heterogeneous Multimedia Information Systems: The Garlic Approach. In *RIDE-DOM*. 124–131. <https://doi.org/10.1109/RIDE.1995.378736>
- [13] A. K. Chandra and P. M. Merlin. 1977. Optimal Implementation of Conjunctive Queries in Relational Data Bases. In *STOC*. 77–90. <https://doi.org/10.1145/800105.803397>
- [14] S. Chaudhuri. 1998. An Overview of Query Optimization in Relational Systems. In *PODS*. 34–43. <https://doi.org/10.1145/275487.275492>
- [15] S. Chaudhuri, V. R. Narasayya, and R. Ramamurthy. 2009. Exact Cardinality Query Optimization for Optimizer Testing. *PVLDB* 2, 1 (2009), 994–1005. <https://doi.org/10.14778/1687627.1687739>
- [16] J. Cheney, L. Chiticariu, and W.-C. Tan. 2009. Provenance in Databases: Why, How, and Where. *Foundations and Trends in Databases* 1, 4 (2009), 379–474. <https://doi.org/10.1561/19000000006>
- [17] Y. Chronis, Y. Wang, Y. Gan, S. Abu-El-Hajja, C. Lin, C. Binnig, and F. Özcan. 2024. CardBench: A Benchmark for Learned Cardinality Estimation in Relational Databases. *arXiv*. [arXiv:2408.16170](https://arxiv.org/abs/2408.16170) <https://arxiv.org/abs/2408.16170> Accessed 2026-06-16.
- [18] S. Chu, B. Murphy, J. Roesch, A. Cheung, and D. Suciu. 2018. Axiomatic Foundations and Algorithms for Deciding Semantic Equivalences of SQL Queries. *PVLDB* 11, 11 (2018), 1482–1495. <https://doi.org/10.14778/3236187.3236200>
- [19] S. Chu, C. Wang, K. Weitz, and A. Cheung. 2017. Cosette: An Automated Prover for SQL. In *CIDR*. 1–10. <https://www.cidrdb.org/cidr2017/papers/p51-chu-cidr17.pdf>
- [20] E. F. Codd. 1970. A Relational Model of Data for Large Shared Data Banks. *Commun. ACM* 13, 6 (1970), 377–387. <https://doi.org/10.1145/362384.362685>
- [21] R. L. Cole and G. Graefe. 1994. Optimization of Dynamic Query Evaluation Plans. In *SIGMOD*. 150–160. <https://doi.org/10.1145/191839.191872>
- [22] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking Cloud Serving Systems with YCSB. In *SoCC*. 143–154. <https://doi.org/10.1145/1807128.1807152>
- [23] Benoit Dageville, Thierry Cruanes, Marcin Zukowski, Vadim Antonov, Artin Avanes, Jon Bock, Jonathan Claybaugh, Daniel Engovatov, Martin Hentschel, Jiansheng Huang, Allison W. Lee, Ashish Motivala, Abdul Q. Munir, Steven Pelley, Peter Povinec, Greg Rahn, Spyridon Triantafyllis, and Philipp Unterbrunner. 2016. The Snowflake Elastic Data Warehouse. In *SIGMOD*. 215–226. <https://doi.org/10.1145/2882903.2903741>
- [24] Jérôme Darmont, Fadila Bentayeb, and Omar Boussaïd. 2007. Benchmarking Data Warehouses. *International Journal of Business Intelligence and Data Mining* 2, 1 (2007), 79–104. <https://doi.org/10.1504/IJBIDM.2007.012947>
- [25] Yuhao Deng, Chengliang Chai, Lei Cao, Qin Yuan, Siyuan Chen, Yanrui Yu, Zhaoze Sun, Junyi Wang, Jiajun Li, Ziqi Cao, Kaisen Jin, Chi Zhang, Yuqing Jiang, Yuanfang Zhang, Yuping Wang, Ye Yuan, Guoren Wang, and Nan Tang. 2024. LakeBench: A Benchmark for Discovering Joinable and Unionable Tables in Data Lakes. *PVLDB* 17, 8 (2024), 1925–1938. <https://doi.org/10.14778/3659437.3659448>
- [26] A. Deshpande, Z. Ives, and V. Raman. 2007. Adaptive Query Processing. *Foundations and Trends in Databases* 1, 1 (2007), 1–140. <https://doi.org/10.1561/19000000001>
- [27] Jennie Duggan, Aaron J. Elmore, Michael Stonebraker, Magda Balazinska, Bill Howe, Jeremy Kepner, Sam Madden, David Maier, Tim Mattson, and Stan Zdonik. 2015. The BigDAWG Polystore System. *SIGMOD Record* 44, 2 (2015), 11–16. <https://doi.org/10.1145/2814710.2814713>
- [28] Orri Erling, Alex Averbuch, Josep Larriba-Pey, Hassan Chafi, Andrey Gubichev, Arnau Prat, Minh-Duc Pham, and Peter Boncz. 2015. The LDBC Social Network Benchmark: Interactive Workload. In *SIGMOD*. 619–630. <https://doi.org/10.1145/2723372.2742786>
- [29] Cesar A. Galindo-Legaria and Arnon Rosenthal. 1997. Outerjoin Simplification and Reordering for Query Optimization. *ACM Transactions on Database Systems* 22, 1 (1997), 43–74. <https://doi.org/10.1145/244810.244812>
- [30] G. Graefe. 1990. Encapsulation of Parallelism in the Volcano Query Processing System. In *SIGMOD*. 102–111. <https://doi.org/10.1145/93597.98720>
- [31] G. Graefe. 1995. The Cascades Framework for Query Optimization. *IEEE Data Engineering Bulletin* 18, 3 (1995), 19–29. <https://www.sigmod.org/publications/dblp/db/journals/debu/Graefe95a.html>
- [32] G. Graefe and W. J. McKenna. 1993. The Volcano Optimizer Generator: Extensibility and Efficient Search. In *ICDE*. 209–218. <https://doi.org/10.1109/ICDE.1993.344061>
- [33] T. J. Green. 2011. Containment of Conjunctive Queries on Annotated Relations. *Theory of Computing Systems* 49, 2 (2011), 429–459. <https://doi.org/10.1007/s00224-011-9327-6>
- [34] T. J. Green, G. Karvounarakis, and V. Tannen. 2007. Provenance Semirings. In *PODS*. 31–40. <https://doi.org/10.1145/1265530.1265535>
- [35] Anurag Gupta, Deepak Agarwal, Derek Tan, Jakub Kulesza, Rahul Pathak, Stefano Stefani, and Vidhya Srinivasan. 2015. Amazon Redshift and the Case for Simpler Data Warehouses. In *SIGMOD*. 1917–1923. <https://doi.org/10.1145/2723372.2742795>
- [36] Yuxing Han, Ziniu Wu, Peizhi Wu, Rong Zhu, Jingyi Yang, Liang Wei Tan, Kai Zeng, Gao Cong, Yanzhao Qin, Andreas Pfadler, Zhengping Qian, Jingren Zhou, Jiangneng Li, and Bin Cui. 2021. Cardinality Estimation in DBMS: A Comprehensive Benchmark Evaluation. *PVLDB* 15, 4 (2021), 752–765. <https://doi.org/10.14778/3503585.3503586>
- [37] B. Hilprecht, A. Schmidt, M. Kulesza, A. Molina, K. Kersting, and C. Binnig. 2020. DeepDB: Learn from Data, Not from Queries! *PVLDB* 13, 7 (2020), 992–1005. <https://doi.org/10.14778/3384345.3384349>
- [38] Y. E. Ioannidis. 1996. Query Optimization. *Comput. Surveys* 28, 1 (1996), 121–123. <https://doi.org/10.1145/234313.234367>
- [39] Z. G. Ives, D. Florescu, M. Friedman, A. Levy, and D. S. Weld. 1999. An Adaptive Query Execution System for Data Integration. *ACM SIGMOD Record* 28, 2 (1999), 299–310. <https://doi.org/10.1145/304181.304209>
- [40] David Justen, Daniel Ritter, Campbell Fraser, Andrew Lamb, Allison Lee, Thomas Bodner, Mhd Yamen Haddad, Steffen Zeuch, Volker Markl, and Matthias Boehm. 2024. POLAR: Adaptive and Non-invasive Join Order Selection via Plans of Least Resistance. *PVLDB* 17, 6 (2024), 1350–1363. <https://doi.org/10.14778/3648160.3648175>
- [41] N. Kabra and D. J. DeWitt. 1998. Efficient Mid-Query Re-Optimization of Sub-Optimal Query Execution Plans. In *SIGMOD*. 106–117. <https://doi.org/10.1145/276304.276315>
- [42] Alfons Kemper and Thomas Neumann. 2011. HyPer: A Hybrid OLTP&OLAP Main Memory Database System Based on Virtual Memory Snapshots. In *ICDE*. 195–206. <https://doi.org/10.1109/ICDE.2011.5767867>
- [43] Andreas Kipf, Thomas Kipf, Bernhard Radke, Viktor Leis, Peter Boncz, and Alfons Kemper. 2019. Learned Cardinalities: Estimating Correlated Joins with Deep Learning. In *CIDR*. 1–11. <https://www.cidrdb.org/cidr2019/papers/p101-kipf-cidr19.pdf>
- [44] Marcel Kornacker, Alexander Behm, Victor Bittorf, Taras Bobrovitsky, Casey Ching, Alan Choi, Justin Erickson, Martin Grund, Daniel Hecht, Matthew Jacobs, Ishaan Joshi, Lenni Kuff, Dileep Kumar, Alex Leblang, Nong Li, Ippokratis Pandis, Henry Robinson, David Rorke, Silvius Rus, John Russell, Dimitris Tsirgiannis, Skye Wanderman-Milne, and Michael Yoder. 2015. Impala: A Modern, Open-Source SQL Engine for Hadoop. In *CIDR*. https://www.cidrdb.org/cidr2015/Papers/CIDR15_Paper28.pdf
- [45] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann. 2015. How Good Are Query Optimizers, Really? *PVLDB* 9, 3 (2015), 204–215. <https://doi.org/10.14778/2850583.2850594>
- [46] V. Leis, B. Radke, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann. 2018. Query Optimization Through the Looking Glass, and What We Found Running the Join Order Benchmark. *Vldb Journal* 27, 5 (2018), 643–668. <https://doi.org/10.1007/s00778-017-0480-7>

- [47] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. 2021. Bao: Making Learned Query Optimization Practical. In *SIGMOD*. 1275–1288. <https://doi.org/10.1145/3448016.3452838>
- [48] R. Marcus and O. Papaemmanouil. 2019. Neo: A Learned Query Optimizer. *PVLDB* 12, 11 (2019), 1705–1718. <https://doi.org/10.14778/3342263.3342644>
- [49] V. Markl, V. Raman, D. Simmen, G. Lohman, H. Pirahesh, and M. Cilimdžić. 2004. Robust Query Processing Through Progressive Optimization. In *SIGMOD*. 659–670. <https://doi.org/10.1145/1007568.1007642>
- [50] Sergey Melnik, Andrey Gubarev, Jing Jing Long, Geoffrey Romer, Shiva Shivakumar, Matt Tolton, and Theo Vassilakis. 2010. Dremel: Interactive Analysis of Web-Scale Datasets. *PVLDB* 3, 1-2 (2010), 330–339. <https://doi.org/10.14778/1920841.1920886>
- [51] G. Moerkotte and T. Neumann. 2006. Analysis of Two Existing and One New Dynamic Programming Algorithm for the Generation of Optimal Bushy Join Trees without Cross Products. In *Vldb*. 930–941. <https://dblp.org/rec/conf/vldb/MoerkotteN06.html>
- [52] Raghunath Othayoth Nambiar and Meikel Poess. 2006. The Making of TPC-DS. In *Vldb*. 1049–1058. <https://www.vldb.org/conf/2006/p1049-othayoth.pdf>
- [53] T. Neumann. 2011. Efficiently Compiling Efficient Query Plans for Modern Hardware. *PVLDB* 4, 9 (2011), 539–550. <https://doi.org/10.14778/2002938.2002940>
- [54] P. O’Neil, E. O’Neil, X. Chen, and S. Revilak. 2009. The Star Schema Benchmark and Augmented Fact Table Indexing. In *TPCTC*. 237–252. https://doi.org/10.1007/978-3-642-10424-4_17
- [55] Andrew Pavlo, Erik Paulson, Alexander Rasin, Daniel J. Abadi, David J. DeWitt, Samuel Madden, and Michael Stonebraker. 2009. A Comparison of Approaches to Large-Scale Data Analysis. In *SIGMOD*. 165–178. <https://doi.org/10.1145/1559845.1559865>
- [56] Meikel Poess and Chris Floyd. 2000. New TPC Benchmarks for Decision Support and Web Commerce. *ACM SIGMOD Record* 29, 4 (2000), 64–71. <https://doi.org/10.1145/369275.369291>
- [57] M. Raasveldt and H. Muehleisen. 2019. DuckDB: An Embeddable Analytical Database. In *SIGMOD*. 1981–1984. <https://doi.org/10.1145/3299869.3320212>
- [58] M. Rigger and Z. Su. 2020. Detecting Optimization Bugs in Database Engines via Non-Optimizing Reference Engine Construction. In *ESEC/FSE*. 1140–1152. <https://doi.org/10.1145/3368089.3409710>
- [59] Manuel Rigger and Zhendong Su. 2020. Testing Database Engines via Pivoted Query Synthesis. In *OSDI*. USENIX Association, 667–682. <https://www.usenix.org/conference/osdi20/presentation/rigger>
- [60] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. 1979. Access Path Selection in a Relational Database Management System. In *SIGMOD*. 23–34. <https://doi.org/10.1145/582095.582099>
- [61] Jeff Shute, Radek Vingralek, Bart Samwel, Ben Handy, Chad Whipkey, Eric Rollins, Mircea Oancea, Kyle Littlefield, David Menestrina, Stephan Ellner, John Cieslewicz, Ian Rae, Traian Stancescu, and Himani Apte. 2013. F1: A Distributed SQL Database That Scales. *PVLDB* 6, 11 (2013), 1068–1079. <https://doi.org/10.14778/2536222.2536232>
- [62] Mohamed A. Soliman, Lyublena Antova, Venkatesh Raghavan, Amr El-Helw, Zhongxian Gu, Entong Shen, George C. Caragea, Carlos Garcia-Alvarado, Foyzur Rahman, Michalis Petropoulos, Florian Waas, Sivaramakrishnan Narayanan, Konstantinos Krikellias, and Rhonda Baldwin. 2014. Orca: A Modular Query Optimizer Architecture for Big Data. In *SIGMOD*. 337–348. <https://doi.org/10.1145/2588555.2595637>
- [63] M. Steinbrunn, G. Moerkotte, and A. Kemper. 1997. Heuristic and Randomized Optimization for the Join Ordering Problem. *Vldb Journal* 6, 3 (1997), 191–208. <https://doi.org/10.1007/s007780050040>
- [64] Michael Stonebraker, Daniel J. Abadi, Adam Batkin, Xuedong Chen, Mitch Cherniack, Miguel Ferreira, Edmond Lau, Amerson Lin, Samuel Madden, Elizabeth J. O’Neil, Patrick E. O’Neil, Alexander Rasin, Nga Tran, and Stanley B. Zdonik. 2005. C-Store: A Column-Oriented DBMS. In *Vldb*. 553–564. <https://doi.org/10.5555/1083592.1083658>
- [65] Ashish Thusoo, Joydeep Sen Sarma, Namit Jain, Zheng Shao, Prasad Chakka, Suresh Anthony, Hao Liu, Pete Wyckoff, and Raghotham Murthy. 2009. Hive: A Warehousing Solution over a Map-Reduce Framework. *PVLDB* 2, 2 (2009), 1626–1629. <https://doi.org/10.14778/1687553.1687609>
- [66] S. Wang, S. Pan, and A. Cheung. 2024. QED: A Powerful Query Equivalence Decider for SQL. *PVLDB* 17, 11 (2024), 3602–3614. <https://doi.org/10.14778/3681954.3682024>
- [67] Z. Wu, P. Negi, M. Alizadeh, T. Kraska, and S. Madden. 2023. FactorJoin: A New Cardinality Estimation Framework for Join Queries. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–27. <https://doi.org/10.1145/3588721>
- [68] Z. Yang, E. Liang, A. Kamsetty, C. Wu, Y. Duan, X. Chen, P. Abbeel, J. M. Hellerstein, S. Krishnan, and I. Stoica. 2019. Deep Unsupervised Cardinality Estimation. *PVLDB* 13, 3 (2019), 279–292. <https://doi.org/10.14778/3368289.3368294>
- [69] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. 2012. Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. In *NSDI*. 15–28. <https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/zaharia>
- [70] Marcin Zukowski, Mark van de Wiel, and Peter Boncz. 2012. Vectorwise: A Vectorized Analytical DBMS. In *ICDE*. 1349–1350. <https://doi.org/10.1109/ICDE.2012.148>